МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
<БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ>

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2

Специальность АС-68

Выполнил
В. А Веремейчук
студент группы АС-67

Проверил
А. А. Крощенко,
доцент кафедры ИИТ,
«___> _____2026 г.

Брест 2026

Цель работы: изучить применение линейной и логистической регрессии для решения практических задач. Научиться обучать модели, оценивать их качество с помощью соответствующих метрик и интерпретировать результаты.

Вариант 3

Задание. выполнить задания по варианту (регрессия и классификация), построить все требуемые визуализации и рассчитать метрики.

Вариант 3

- **Регрессия (Прогнозирование расхода топлива)**
  1. Auto MPG
  2. Предсказать расход топлива (mpg)
  3. **Задания:**
     - загрузите данные, обработайте пропуски и категориальные признаки;
     - обучите модель **линейной регрессии**, используя в качестве признаков cylinders, horsepower, weight;
     - рассчитайте **MSE** и **R^2**;
     - визуализируйте зависимость mpg от horsepower с линией регрессии.
- **Классификация (Диагностика диабета)**

[redacted]

ОСНОВЫ МАШИННОГО ОБУЧЕНИЯ, ЛР № 2, 2025

1. Pima Indians Diabetes
2. Предсказать наличие диабета (Outcome)
3. **Задания:**
   - загрузите данные, выполните стандартизацию признаков;
   - обучите модель **логистической регрессии**;
   - рассчитайте **Accuracy**, **Precision** и **Recall**;
   - постройте **матрицу ошибок** и сделайте выводы о количестве ложноположительных и ложноотрицательных срабатываний.

Выполнение:

Код программы

```
import os

import urllib.request
```

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    mean_squared_error,
    precision_score,
    r2_score,
    recall_score,
)
from sklearn.model_selection import cross_val_predict, train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
AUTO_PATH = os.path.join(BASE_DIR, "auto-mpg.csv")
DIAB_PATH = os.path.join(BASE_DIR, "pima-indians-diabetes.csv")

AUTO_URL = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/mpg.csv"
DIAB_URL = ("https://raw.githubusercontent.com/jbrownlee/Datasets/"
            "master/pima-indians-diabetes.data.csv")

if not os.path.exists(AUTO_PATH):
    print("Скачиваем auto-mpg.csv ...")
    urllib.request.urlretrieve(AUTO_URL, AUTO_PATH)

if not os.path.exists(DIAB_PATH):
    print("Скачиваем pima-indians-diabetes.csv ...")
    urllib.request.urlretrieve(DIAB_URL, DIAB_PATH)

print("\n  Запуск задачи регрессии (Auto MPG) ")

df_auto = pd.read_csv(AUTO_PATH)
```

```
df_auto["horsepower"] = pd.to_numeric(df_auto["horsepower"], errors="coerce")
df_auto = df_auto.dropna(subset=["mpg", "cylinders", "horsepower", "weight"])

X_reg = df_auto[["cylinders", "horsepower", "weight"]]
y_reg = df_auto["mpg"]

X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)

lin_reg = LinearRegression()
lin_reg.fit(X_train_r, y_train_r)

y_pred_r = lin_reg.predict(X_test_r)
mse = mean_squared_error(y_test_r, y_pred_r)
r2 = r2_score(y_test_r, y_pred_r)

print(f"\nКоэффициенты модели:")
for name, coef in zip(X_reg.columns, lin_reg.coef_):
    print(f"  {name:12s}: {coef:+.4f}")
print(f"  intercept   : {lin_reg.intercept_:+.4f}")

print(f"\nMSE: {mse:.4f}")
print(f"R^2: {r2:.4f}")

plt.figure(figsize=(8, 6))
plt.scatter(
    df_auto["horsepower"],
    df_auto["mpg"],
    alpha=0.6,
    color="blue",
    label="Данные",
)
m, b = np.polyfit(df_auto["horsepower"], df_auto["mpg"], 1)
```

```
plt.plot(
    df_auto["horsepower"],
    m * df_auto["horsepower"] + b,
    color="red",
    label="Линия регрессии",
)
plt.title(
    "Зависимость расхода топлива (mpg) от лошадиных сил (horsepower)",
    fontsize=12,
)
plt.xlabel("Horsepower", fontsize=10)
plt.ylabel("MPG", fontsize=10)
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig(os.path.join(BASE_DIR, "mpg_horsepower_regression.png"), dpi=300)
plt.show()

print("\n  Запуск задачи классификации (Pima Diabetes) ")

col_names = [
    "pregnancies",
    "glucose",
    "blood_pressure",
    "skin_thickness",
    "insulin",
    "bmi",
    "pedigree",
    "age",
    "Outcome",
]

df_diabetes = pd.read_csv(
    DIAB_PATH, comment="#", header=None, names=col_names
)
```

```
X_clf = df_diabetes.drop("Outcome", axis=1)
y_clf = df_diabetes["Outcome"]

model_pipeline = make_pipeline(
   StandardScaler(),
   LogisticRegression(random_state=42, max_iter=1000),
)

y_pred_c = cross_val_predict(model_pipeline, X_clf, y_clf, cv=10)

accuracy = accuracy_score(y_clf, y_pred_c)
precision = precision_score(y_clf, y_pred_c)
recall = recall_score(y_clf, y_pred_c)
cm = confusion_matrix(y_clf, y_pred_c)

print(f"\nAccuracy  (кросс-валидация, 10 фолдов): {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall:    {recall:.4f}")

tn, fp, fn, tp = cm.ravel()
print("\n    Анализ матрицы ошибок (по всем 768 строкам)")
print(f"Истинно отрицательные (TN): {tn}")
print(f"Ложноположительные  (FP): {fp}")
print(f"Ложноотрицательные  (FN): {fn}")
print(f"Истинно положительные (TP): {tp}")
print(f"Сумма элементов в матрице: {cm.sum()}")

# ---- Матрица ошибок ----
plt.figure(figsize=(6, 5))
plt.imshow(cm, interpolation="nearest", cmap=plt.cm.Blues)
plt.title("Матрица ошибок — Диагностика диабета", fontsize=12)
plt.colorbar()
tick_marks = np.arange(2)
plt.xticks(tick_marks, ["Нет диабета", "Диабет"])
```

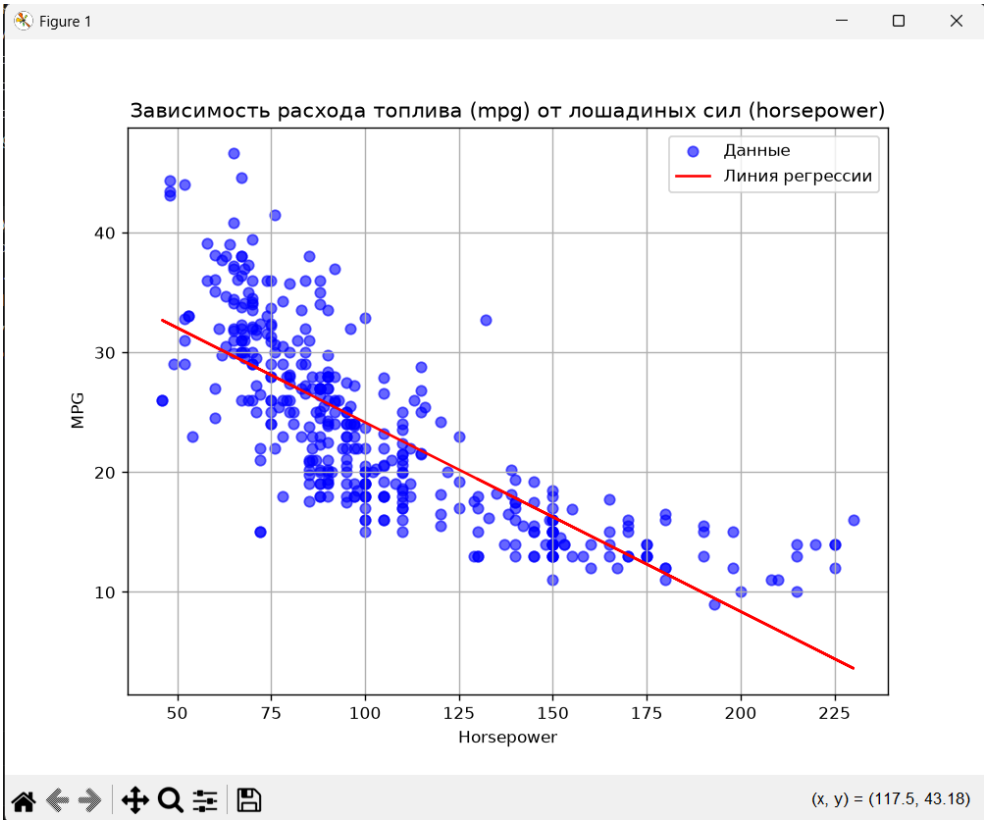
```
plt.yticks(tick_marks, ["Нет диабета", "Диабет"])


thresh = cm.max() / 2.0
for i in range(cm.shape[0]):
   for j in range(cm.shape[1]):
      plt.text(
         j,
         i,
         format(cm[i, j], "d"),
         horizontalalignment="center",
         color="white" if cm[i, j] > thresh else "black",
      )

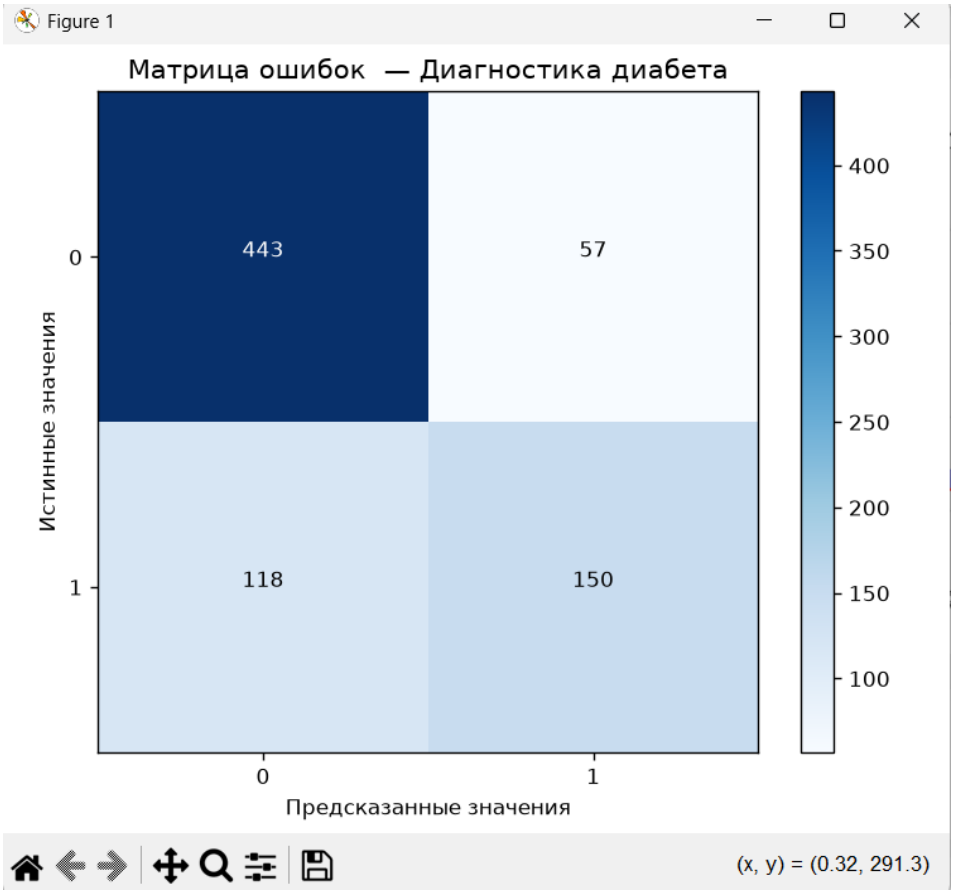

plt.xlabel("Предсказанные значения", fontsize=10)
plt.ylabel("Истинные значения", fontsize=10)
plt.tight_layout()
plt.savefig(os.path.join(BASE_DIR, "diabetes_confusion_matrix.png"), dpi=300)
plt.show()


print("\nГотово. Все картинки сохранены рядом со скриптом.")
```

Результат:



Запуск задачи регрессии (Auto MPG)
MSE: 17.6772
R^2: 0.6537

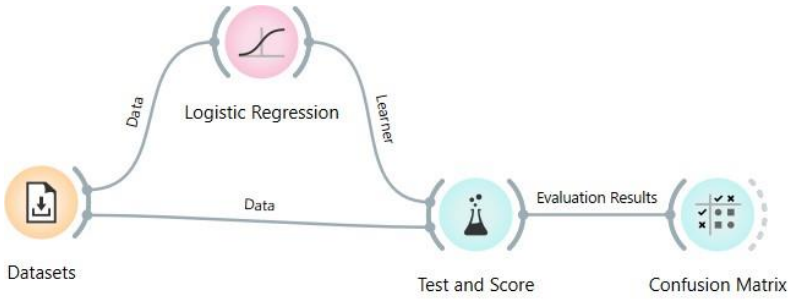


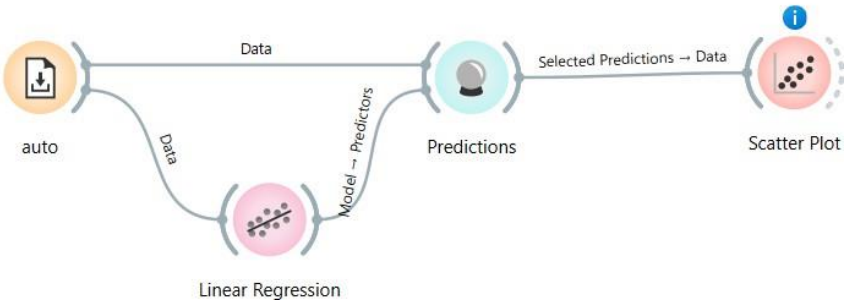
Запуск задачи классификации
Accuracy (на всех данных): 0.7721
Precision: 0.7246
Recall: 0.5597

Анализ матрицы ошибок (по всем 768 строкам)
Истинно отрицательные (TN): 443

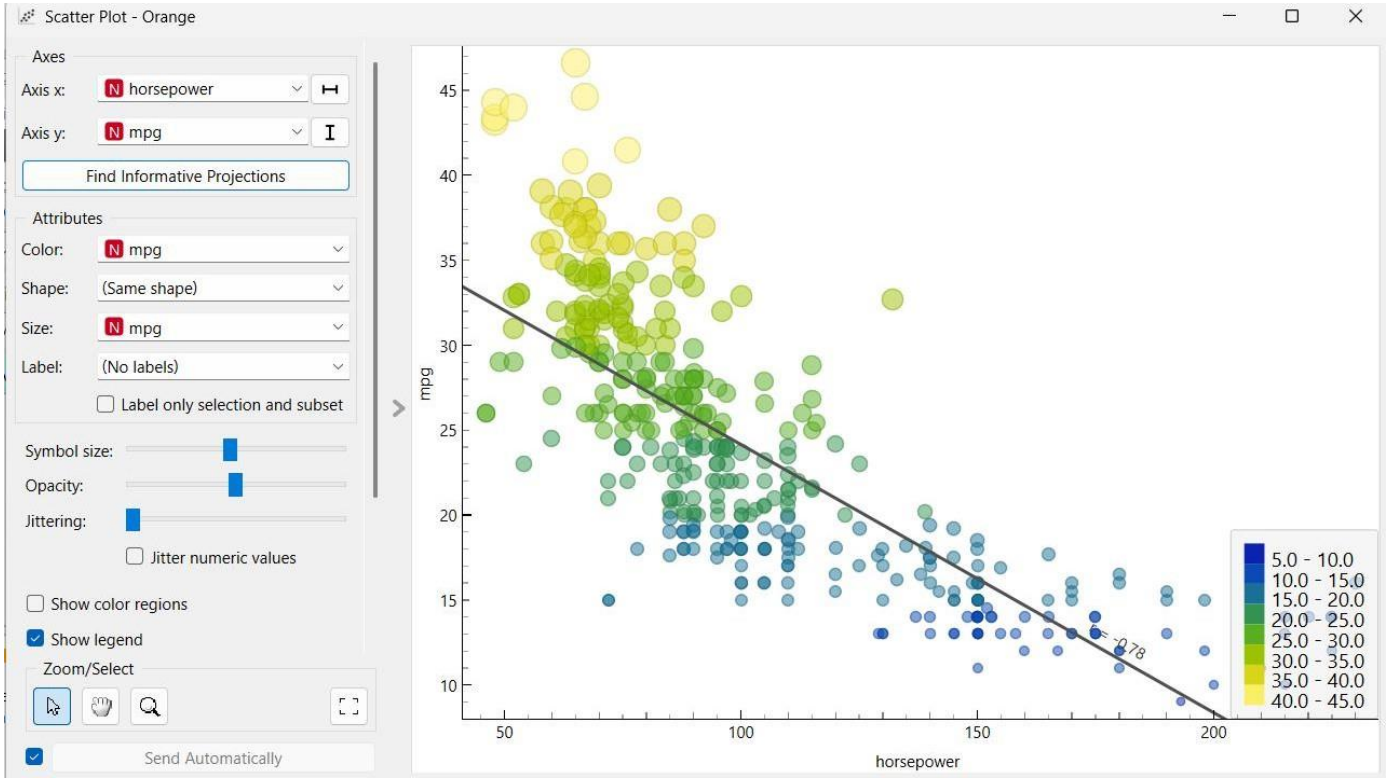
Ложноположительные (FP): 57
Ложноотрицательные (FN): 118
Истинно положительные (TP): 150
Сумма элементов в матрице: 768

Результаты в Orange:





		Predicted: Non-Diabetic	Predicted: Diabetic	Σ
Actual	Non-Diabetic	441	59	500
	Diabetic	113	155	268
	Σ	554	214	768



Вывод: в ходе выполнения лабораторной работы были успешно освоены методы линейной и логистической регрессии для решения практических задач. Были обучены модели для задач регрессии и классификации, рассчитаны ключевые метрики их качества и проанализированы полученные результаты.